

EfficientNet-B0 학습 파이프라인 사용설명서

노후 건물 균열 3등급(우수/보통/불량) 분류 · MobileNetV2와 F1 스코어 성능 비교용

1. 개요

이 파이프라인은 팀원이 만든 MobileNetV2 모델과 성능(F1 스코어)을 비교하기 위해 EfficientNet-B0로 같은 데이터를 학습하는 코드입니다. 공정한 비교를 위해 전처리 과정과 데이터 분리(train/validation/test)를 팀원 코드와 완전히 동일하게 맞췄습니다.

팀원 코드와 같은 것:

- 전처리: 224×224 리사이즈 → (train만) 좌우반전·밝기/대비 증감 → float32 스케일 → ImageNet 정규화
- 데이터 분리: 팀원이 만든 metadata_split.csv를 그대로 사용 (재분리하지 않음)
- 클래스 불균형 가중치 [3.79, 2.95, 0.42], Adam 옵티마이저, 2단계 학습(baseline → 파인튜닝)
- 최고 모델 선택 기준: validation 정확도 (팀원과 동일한 기준으로 학습해야 공정 비교)

다른 것:

- 모델: MobileNetV2 → EfficientNet-B0 (둘 다 ImageNet 사전학습, 224×224 입력)
- 배치 크기: 16 → 128 (20GB VRAM 기준)

최종 비교 지표: 프로젝트 목표가 '불량 탐지'이므로 불량을 positive로 본 불량 F1-score가 핵심 지표입니다. 두 모델의 Test 평가 결과에서 불량 행의 f1-score(및 recall/precision)를 비교합니다.

※ 팀원의 기존 파일(src/ 바로 아래)은 일절 수정하지 않았습니다. 새 코드는 전부 src/efficientnet/ 폴더 안에만 있습니다.

2. 파일 구성

파일	역할
preprocess_efficientnet.py	데이터셋/데이터로더 정의. CSV의 팀원 PC 경로를 data/processed_images/ 경로로 바꿔서 읽음. 다른 스크립트가 import해서 사용.
train_efficientnet.py	1단계 baseline 학습. 특징 추출부는 동결하고 분류층만 3등급으로 학습. → model/best_efficientnet_b0_baseline.pth
fine_tune_efficientnet.py	2단계 파인튜닝. 마지막 특징 블록까지 동결을 풀고 작은 학습률로 추가 학습. → model/best_efficientnet_b0_finetuned.pth
evaluate_efficientnet.py	Test 데이터 최종 평가. 정확도, 등급별 F1, macro F1, 혼동행렬 CSV 출력.

3. 사전 준비 (런팟 기준)

3-1. 프로젝트 받기

```
cd /workspace
git clone https://github.com/gpwhdqkr/hn_old-building.git
cd hn_old-building
```

3-2. 라이브러리 설치

```
pip install torch torchvision pandas pillow scikit-learn
```

런팍 PyTorch 템플릿을 쓰면 torch/torchvision은 보통 이미 설치되어 있습니다. 그 경우 나머지만 설치하면 됩니다: `pip install pandas pillow scikit-learn`

3-3. 데이터 배치

이미지 폴더는 용량 문제로 깃에 없습니다. 구글드라이브에서 받은 224×224 변환 이미지를 아래 위치에 풀어야 합니다. (metadata_split.csv는 깃에 포함되어 있어 클론하면 따라옵니다)

```
/workspace/hn_old-building/data/processed_images/ ← 이미지 37,285장  
/workspace/hn_old-building/data/processed/metadata_split.csv ← 깃에 포함
```

확인 방법: 아래 명령의 결과가 37285면 정상입니다.

```
find data/processed_images -type f | wc -l
```

4. 실행 순서

프로젝트 폴더(/workspace/hn_old-building)에서 아래 순서대로 실행합니다.

STEP 0. 데이터 로딩 확인 (선택이지만 권장)

```
python src/efficientnet/preprocess_efficientnet.py
```

이미지 묶음 형태가 [128, 3, 224, 224]로 출력되면 데이터 연결이 정상입니다. 여기서 에러가 나면 6장(문제 해결)을 확인하세요.

실험 이름(RUN_NAME): epoch 수 등 설정을 바꿔가며 여러 번 실험할 때는 명령 앞에 RUN_NAME=이름 을 붙이세요. 결과 파일명에 이름이 붙어서 이전 실험을 덮어쓰지 않고 나란히 쌓입니다. 미지정 시 "default"가 쓰이며, 같은 이름으로 다시 돌리면 그 실험 결과만 덮어씹니다. STEP 1~3은 반드시 같은 RUN_NAME을 사용해야 합니다 (예: RUN_NAME=epoch20).

STEP 1. baseline 학습 (분류충만)

```
RUN_NAME=epoch10 python src/efficientnet/train_efficientnet.py
```

10 epoch 학습하며, validation 정확도가 가장 높았던 시점의 모델이 model/best_efficientnet_b0_baseline_<실험이름>.pth 로 저장됩니다. (매 epoch 불량 F1도 참고용으로 출력됨)

STEP 2. 파인튜닝 (마지막 특징 블록까지)

```
RUN_NAME=epoch10 python src/efficientnet/fine_tune_efficientnet.py
```

baseline 모델을 불러와 추가 학습합니다. 최대 10 epoch, validation 정확도가 3회 연속 개선되지 않으면 자동으로 조기 종료됩니다. 결과는 model/best_efficientnet_b0_finetuned_<실험이름>.pth 로 저장됩니다. (한 번도 개선되지 않아도 baseline 상태가 저장되어 있으므로 파일은 항상 존재합니다)

STEP 3. Test 평가 (F1 스코어 산출)

```
RUN_NAME=epoch10 python src/efficientnet/evaluate_efficientnet.py
```

기본값은 파인튜닝 모델 평가입니다. baseline을 평가하려면 evaluate_efficientnet.py 파일 상단의 변수를 다음과 같이 바꾼 뒤 다시 실행하세요.

```
EVALUATION_TARGET = "baseline" # 또는 "finetuned"
```

5. 결과 확인과 MobileNetV2 비교 방법

STEP 3 실행 시 화면에 다음이 출력됩니다.

- Test 정확도 (전체 중 맞힌 비율)
- 등급별 성능표: 우수/보통/불량 각각의 precision, recall, F1-score
- 혼동행렬: 실제 등급별로 어떤 등급으로 예측했는지 개수 표 (CSV로도 저장됨: test_results/efficientnet_b0/<실험이름>/test_confusion_matrix_*.csv — 폴더 자동 생성)
- [비교용 최종 지표] Test 불량 F1 ← 이 숫자가 핵심 비교 기준 (불량 = positive)

비교 방법: 팀원의 evaluate_model.py 출력에도 같은 '등급별 성능' 표가 나옵니다. 두 출력에서 '불량' 행의 f1-score를 비교하는 것이 핵심이고, 불량 행의 recall(실제 불량을 놓치지 않는 비율)과 precision(불량 판정의 신뢰도), macro avg도 함께 보면 좋습니다. 두 모델 모두 같은 test set(5,558장)으로 평가되므로 공정한 비교입니다.

6. 문제 해결 (자주 나는 에러)

■ metadata_split.csv를 찾을 수 없습니다

깃 클론이 제대로 안 됐거나 다른 폴더에서 실행한 경우입니다. /workspace/hn_old-building 에서 실행하는지, data/processed/metadata_split.csv가 있는지 확인하세요.

■ 이미지 폴더를 찾을 수 없습니다

구글드라이브의 이미지가 data/processed_images/ 위치에 없는 경우입니다. 압축을 푼 뒤 폴더 구조가 data/processed_images/TS_아파트/... 형태인지 확인하세요. (중간에 폴더가 한 겹 더 생기지 않았는지 주의)

■ CUDA out of memory (VRAM 부족)

preprocess_efficientnet.py 상단의 batch_size = 128 을 64나 32로 줄이고 다시 실행하세요. 배치 크기는 학습 속도에만 영향을 주고 비교의 공정성에는 문제없습니다.

■ baseline 모델을 찾을 수 없습니다

STEP 1(train)을 건너뛰고 STEP 2(fine_tune)를 실행한 경우입니다. 반드시 train_efficientnet.py 를 먼저 실행하세요.

■ DataLoader 관련 worker 에러

preprocess_efficientnet.py 상단의 num_workers = 4 를 0으로 바꾸면 해결됩니다. (속도는 느려지지만 결과는 동일)

■ GPU가 아니라 CPU로 학습됨 ('사용 장치 : cpu' 출력)

torch가 CPU 버전으로 설치된 경우입니다. 런타임 GPU 팟이라면 pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126 으로 재설치하세요.

7. 주요 설정값 변경 위치

설정	파일	변수
배치 크기 (기본 128)	preprocess_efficientnet.py	batch_size
데이터 로딩 워커 수 (기본 4)	preprocess_efficientnet.py	num_workers
baseline 학습 epoch (기본 10)	train_efficientnet.py	num_epochs
파인튜닝 최대 epoch (기본 10)	fine_tune_efficientnet.py	num_epochs
조기 종료 기준 (기본 3회)	fine_tune_efficientnet.py	patience
평가 대상 모델	evaluate_efficientnet.py	EVALUATION_TARGET
실험 이름 (결과 파일 구분)	실행 시 환경변수로 지정	RUN_NAME

작성일: 2026-07-23 · 대상 코드: src/efficientnet/ (4개 파일) · 실행 환경: RunPod Ubuntu, GPU 20GB VRAM 기준